

Введение

В современном программировании важную роль играет правильная архитектура приложений. Для этого используются шаблоны проектирования (design patterns), которые позволяют создавать гибкие, расширяемые и удобные в поддержке системы.

Одним из таких шаблонов является **Абстрактная фабрика (Abstract Factory)**. В данном практическом задании была реализована программа на языке Python, демонстрирующая работу этого паттерна на примере компании, в которой сотрудники используют различные устройства и транспорт.

Основная задача работы — изучить принципы создания семейств взаимосвязанных объектов без привязки к их конкретным классам.

Теоретическая часть

Паттерн **Абстрактная фабрика** — это порождающий шаблон проектирования, который предоставляет интерфейс для создания семейств взаимосвязанных объектов без указания их конкретных классов.

Основная идея заключается в том, что фабрика создает не один объект, а сразу группу связанных объектов, которые должны использоваться вместе. Например, сотрудник компании получает не только устройство для работы, но и служебный транспорт.

Преимущества данного подхода:

- упрощение создания групп объектов;
- обеспечение совместимости объектов одного семейства;
- возможность легко добавлять новые типы объектов;
- снижение зависимости кода от конкретных классов.

Недостатком является увеличение количества классов и усложнение структуры программы.

Постановка задачи

Согласно условию задания, необходимо реализовать систему, в которой:

- существует компания с сотрудниками;
- есть два типа сотрудников: программист и директор;

- каждый сотрудник имеет:
 - рабочее устройство (компьютер или ноутбук);
 - служебный автомобиль (BMW или LADA).

Для этого требуется создать:

- абстрактные классы устройств и транспорта;
- конкретные реализации (Laptop, Computer, BMW, LADA);
- фабрики для создания сотрудников;
- класс сотрудника, который получает объекты через фабрику

Заключение

В ходе выполнения практической работы был изучен и реализован паттерн «Абстрактная фабрика». Данный паттерн позволяет создавать семейства взаимосвязанных объектов, обеспечивая их совместимость и независимость от конкретных реализаций.

Практическая реализация показала, что использование данного шаблона значительно упрощает расширение программы и улучшает ее архитектуру.

Полученные знания могут быть применены при разработке сложных программных систем, где требуется гибкость и масштабируемость.

```
from abc import ABC, abstractmethod
```

```
# =====
# Абстрактный класс служебного автомобиля
# =====
class WorkingCar(ABC):
    def __init__(self, model: str, price: float, year: int):
        self.model = model
        self.price = price
        self.year = year

    @abstractmethod
    def get_tax_cost(self) -> float:
        pass

    @abstractmethod
    def get_info(self) -> str:
```

pass

=====

Абстрактный класс рабочего устройства

=====

class WorkingDevice(ABC):

def __init__(self, model: str, price: float):

self.model = model

self.price = price

@abstractmethod

def get_accessories_cost(self) -> float:

pass

@abstractmethod

def get_info(self) -> str:

pass

=====

Конкретные автомобили

=====

class LADA(WorkingCar):

def __init__(self, model: str, price: float, year: int):

super().__init__(model, price, year)

def get_tax_cost(self) -> float:

return self.price * 0.01

def get_info(self) -> str:

return (

f"Автомобиль: LADA\n"

f"Модель: {self.model}\n"

f"Цена: {self.price} тр\n"

f"Год выпуска: {self.year}\n"

f"Стоимость налога: {self.get_tax_cost()} тр"

)

class BMW(WorkingCar):

def __init__(self, model: str, price: float, year: int):

super().__init__(model, price, year)

def get_tax_cost(self) -> float:

return self.price * 0.03

def get_info(self) -> str:

```

        return (
            f"Автомобиль: BMW\n"
            f"Модель: {self.model}\n"
            f"Цена: {self.price} тг\n"
            f"Год выпуска: {self.year}\n"
            f"Стоимость налога: {self.get_tax_cost()} тг"
        )

# =====
# Конкретные рабочие устройства
# =====
class Laptop(WorkingDevice):
    def __init__(self, model: str, price: float):
        super().__init__(model, price)

    def get_accessories_cost(self) -> float:
        return 25000 # зарядка, мышка, сумка

    def get_info(self) -> str:
        return (
            f"Устройство: Ноутбук\n"
            f"Модель: {self.model}\n"
            f"Цена: {self.price} тг\n"
            f"Стоимость аксессуаров: {self.get_accessories_cost()} тг"
        )

class Computer(WorkingDevice):
    def __init__(self, model: str, price: float):
        super().__init__(model, price)

    def get_accessories_cost(self) -> float:
        return 40000 # клавиатура, мышь, колонки и т.д.

    def get_info(self) -> str:
        return (
            f"Устройство: Компьютер\n"
            f"Модель: {self.model}\n"
            f"Цена: {self.price} тг\n"
            f"Стоимость аксессуаров: {self.get_accessories_cost()} тг"
        )

# =====
# Интерфейс фабрики сотрудника
# =====
class IWorkerFactory(ABC):

```

```
@abstractmethod
def create_working_car(self) -> WorkingCar:
    pass
```

```
@abstractmethod
def create_working_device(self) -> WorkingDevice:
    pass
```

```
# =====
# Фабрика программиста
# =====
class ProgrammerFactory(IWorkerFactory):
    def create_working_car(self) -> WorkingCar:
        return LADA("LADA Granta", 6500000, 2021)

    def create_working_device(self) -> WorkingDevice:
        return Laptop("Lenovo IdeaPad 3", 320000)
```

```
# =====
# Фабрика директора
# =====
class DirectorFactory(IWorkerFactory):
    def create_working_car(self) -> WorkingCar:
        return BMW("BMW X5", 28000000, 2023)

    def create_working_device(self) -> WorkingDevice:
        return Computer("HP ProDesk", 540000)
```

```
# =====
# Класс сотрудника
# =====
class Worker:
    def __init__(self, worker_type: str, factory: IWorkerFactory):
        self.worker_type = worker_type
        self.working_car = factory.create_working_car()
        self.working_device = factory.create_working_device()

    def show_info(self):
        print(f"Тип сотрудника: {self.worker_type}")
        print("-" * 40)
        print(self.working_car.get_info())
        print("-" * 40)
        print(self.working_device.get_info())
        print("=" * 50)
```

```

# =====
# Главная программа
# =====
if __name__ == "__main__":
    programmer = Worker("Программист", ProgrammerFactory())
    director = Worker("Директор", DirectorFactory())

    print("Информация о программисте:")
    programmer.show_info()

    print("Информация о директоре:")
    director.show_info()
Информация о программисте:
Тип сотрудника: Программист
-----
Автомобиль: LADA
Модель: LADA Granta
Цена: 6500000 тг
Год выпуска: 2021
Стоимость налога: 65000.0 тг
-----
Устройство: Ноутбук
Модель: Lenovo IdeaPad 3
Цена: 320000 тг
Стоимость аксессуаров: 25000 тг
=====

Информация о директоре:
Тип сотрудника: Директор
-----
Автомобиль: BMW
Модель: BMW X5
Цена: 28000000 тг
Год выпуска: 2023
Стоимость налога: 840000.0 тг
-----
Устройство: Компьютер
Модель: HP ProDesk
Цена: 540000 тг
Стоимость аксессуаров: 40000 тг
=====

```